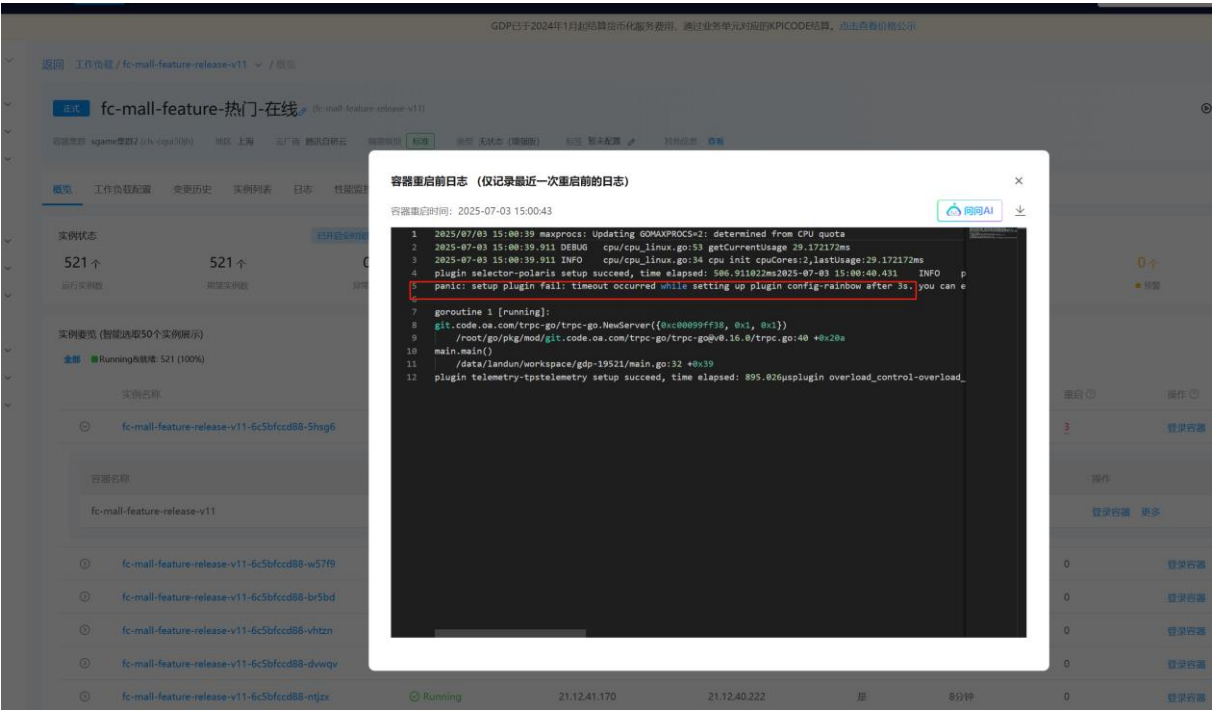


20250707 日报

今日工作内容

1. 大致解决了初始化插件超时而导致容器重启几次的问题。

现象：在晚高峰时期（特别是新赛季更新前几天），初始化插件时出现超时现象（trpc-go 的插件超时时间默认为 3 秒）：



原因：

- 1、域名访问会有随机性，可能存在跨地访问，这个比较影响耗时；
- 2、再就是可能会有网络抖动，也会影响。

目前解决方案：

- 1、域名访问会有随机性，可能存在跨地访问，这个比较耗时，因此可以改为用北极星就近访问，对耗时会有改善:<http://api.rainbow.oa.com:8080>->polaris://65026305:65536;
- 2、直接改 trpc-config 中的插件超时配置信息：
 - 代码中直接修改全局变量

```
const pluginSetupTimeout = time.Second * 10

// 修改每个插件的超时时间
func initTRPCConfig() {
    plugin.SetupTimeout = pluginSetupTimeout
}

// 注意：上述方法设置超时时间需要在 NewServer 之前，因为初始化插件是在
NewServer 中进行的
```

- yaml 配置信息中直接改

```
global:
    plugin_setup_timeout: ***
```

注意：上面两种方法都是对每个插件设置相同的超时时间的修改方法，如果想只改某个插件，那么可以在该插件对应的 yaml 配置中添加配置项，或者在代码中对 GetSetupHook() 的 key 进行判断，再添加对应修改超时时间逻辑即可。

3、将超时时间在 redis 中存取，从而达到自动控制超时时间的目的，即每次超时就会自动将超时时间乘以 2，然后再将其存入 redis，以供下次读取：

```
1.
// proxies/redis_client.go
var SilkSongPluginTimeoutRedis =
redis.NewClientProxy("trpc.redis.sliksong_plugintimeout.service")
```

```
2.
// setplugintimeout/timeoutprocess.go
const maxTimeout = time.Minute

func Init() {
    // 从 redis 中获取超时时间
    SetupTimeout, err :=
redis.String(proxies.SilkSongPluginTimeoutRedis.Do(context.Background(), "GET", "timeout"))
    fmt.Println(SetupTimeout)
    if err == nil && SetupTimeout != "" { // 键存在且无错误, 否则超时
        时间是默认值
        fmt.Println("SetupTimeout:", SetupTimeout)
        val, _ := strconv.Atoi(SetupTimeout)
        plugin.SetupTimeout = time.Duration(val) * time.Second
    }
}
```

```
// 超时一定时间后, 使用降级策略处理错误, pluginKey 为插件名称
func Process(pluginKey string) {
    plugin.RegisterSetupHook(pluginKey, func(setup func() error)
error {
        ch := make(chan error)
        go func() { ch <- setup() }()
        select {
        case err := <-ch:
            return err // 正常完成, 返回错误或 nil
        case <-time.After(plugin.SetupTimeout):
            timeoutCallback() // 调用钩子函数
        }
        return nil
    })
}
```

```
// 获取当前设置的超时时间
func GetCurrentTimeout() time.Duration {
```

本地测试结果：

- 一开始 redis 中没有超时时间对应的键值对：

```
[root@drewjjli-1t70osepea workspace]# redis-cli
127.0.0.1:6379> keys *
(empty list or set)
127.0.0.1:6379> 
```

- 当拉取七彩石配置超时，此时 redis 中就会自动存入对应的超时时间，其值等于默认超时时间的 2 倍=6 秒：

```
2020-08-11 10:10:10.100 [ERROR] setup plugin fail: timeout occurred while setting up plugin config-rainbow after 3s, you can edit the plugin.SetupTimeout (or global.plugin_setup_timeout in trpc-go.yaml) to increase
goroutine 1 [running]:
  git.code.oa.com/trpc-go/trpc-go.NewServer({0x0, 0x0, 0x0})
    /root/go/pkg/mod/git.code.oa.com/trpc-go/trpc-go@v0.20.0/trpc-go:51 +0x245
  main.main()
    /data/workspace/test/server/main.go:18 +0x1e
```

```
127.0.0.1:6379> keys *
1) "timeout"
127.0.0.1:6379> get timeout
"6"
127.0.0.1:6379> 
```

方法 3 主要是为了处理以下两种情况：

- 对于一个服务，它的超时时间动态变化幅度较大；
- 对于一个服务，事先不清楚它的超时时间，这样可以在重启的过程中自动一次一次的更新 redis 中的超时时间值。

总的来说，对于插件初始化超时的处理办法：

- 可以将七彩石的接入方式的域名从 oa 改到 woa 和改为北极星接入，建议改为北极星方式，耗时更短；
- 实在不行就走修改超时时间进行兜底，这里也有两种方式：一种是简单的直接修改服务的超时时间（首选方案，超时时间不会大幅度动态变化）；还有一种就是把每个服务调整后的超时时间存入 redis，此类方法是用于应对超时时间动态变化和不知道具体超时时间值的情况。

明日待办：

1. 七彩石平台如果挂了的备用配置读取手段的研究。